uProcessWatch and uLoadWatch

Process and load monitoring utilities.

Included MIT MOOS-IvP PDF sources:

- app_uprocwatch.pdf
- app_uloadwatch.pdf

uProcessWatch: Monitoring MOOS Application Health

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1 Overview

The uProcessWatch application monitors the health of a set of MOOS application. It does two things:

- It monitors the presence of a set of MOOS apps,
- It monitors the CPU load of a set of MOOS apps.

In the case of the former, uProcessWatch continually monitors the DB_CLIENTS list for applications it has responsibility for watching. In the case of the latter, MOOS apps are already monitoring and reporting their own CPU load and uProcessWatch is doing nothing more than gathering that information for display in the terminal or appcast message. An example terminal output is shown below:

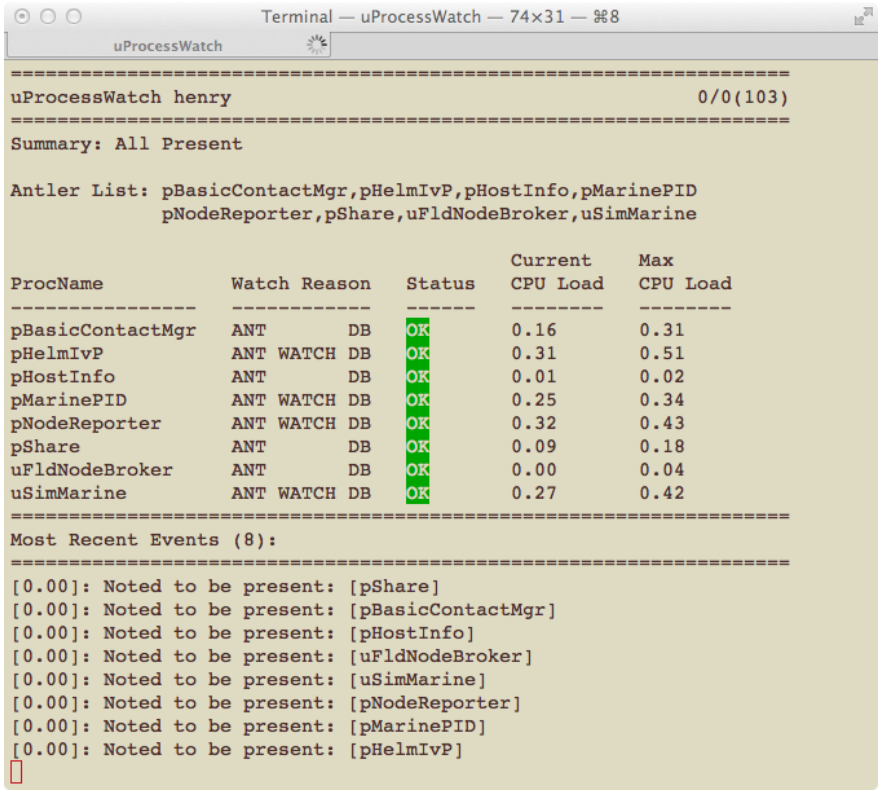

Figure 1: **A typical terminal or appcast report for `uProcessWatch`.**

The first line, "Summary: All Present", tells you that no processes are missing. This is also posted in the variable `PROC_WATCH_SUMMARY`. The list of applications launched with `pAntler` is shown in the next block. The main body shows, for each watched application, the reason why the process is on the watch list, the status, current CPU load as reported by the process itself, and the maximum CPU load noted so far.

The bottom section of the terminal output shows the events (similar to all appcasting output). In the case of `uProcessWatch`, events note either the arrival or disappearance of a watched process. Each event also results in the posting of the variable `PROC_WATCH_EVENT`. The user may configure `uProcessWatch` to *not* watch certain named applications or patterns of applications, such as `uXMS*`, to avoid unwarranted alerts.

2 Typical uProcessWatch Usage Scenarios

2.1 Using uProcessWatch with AppCasting and pMarineViewer

The first usage scenario is perhaps the most typical since it is viable both in simulation and in the field where the vehicles have a network connection to a shoreside MOOS community. The idea is shown in Figure 2 below. Multiple vehicles each run `uProcessWatch` locally. A network connection from each vehicle to a shoreside MOOS community is used to share appcast messages using `pShare`. The shoreside community is running `pMarineViewer` which supports a multi-vehicle appcast viewing

mode. In this way, the shoreside user may monitor the health of all vehicles' applications with one tool. If a process on a single vehicle goes missing, the menu item on the shoreside viewer for that vehicle turns red.

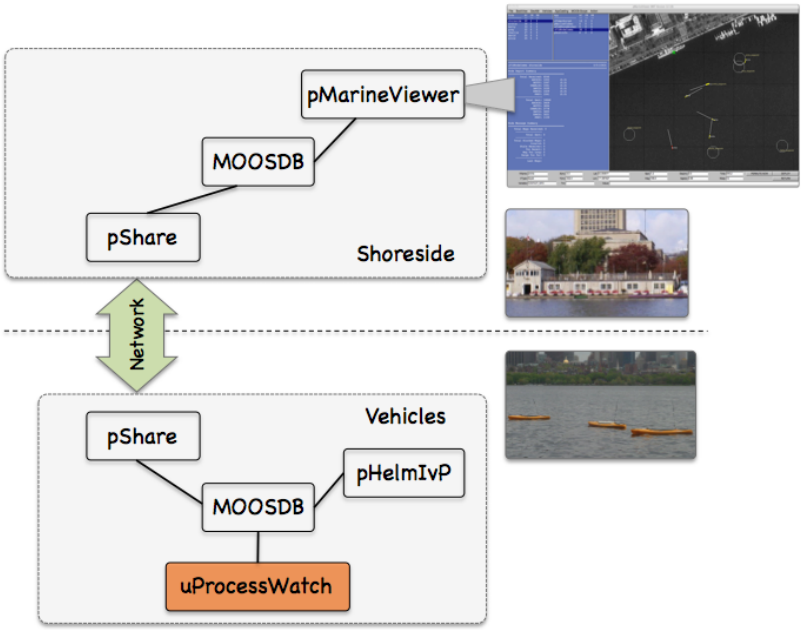

Figure 2: **Using uProcessWatch with AppCasting:** uProcessWatch is run locally on each fielded vehicle, generating appcasts posted to the local MOOSDB. Appcasts are shared to the shoreside and collected by the pMarineViewer tool for monitoring processes across all vehicle. This scenario is relevant when there is a network connection from the vehicles to the shoreside.

2.2 Directly Accessing the PROC_WATCH_SUMMARY Output

The main health indicator produced by uProcessWatch in the MOOS variable PROC_WATCH_SUMMRY. This can be checked on a remote machine by simply scoping on this variable. The uMS application distributed with MOOS will do the trick for example. To focus solely on this variable, the uXMS tool may also be used as follows:

```
$ uXMS --server_host=10.25.0.72 --server_port=9000 PROC_WATCH_SUMMARY
```

Or one may ssh onto the vehicle and launch a scope locally on this variable.

3 Using and Configuring the uProcessWatch Utility

The primary configuration of uProcessWatch is defining the *watch list*, the set of other MOOS processes to monitor. By default all processes ever noted to be connected to the MOOSDB are on the watch list. The same with all processes name in the Antler configuration block of the mission file. This default is used because it is simple, and a good rule of thumb is that if any process disconnects from the MOOSDB, it's probably a sign of trouble.

3.1 The DB_CLIENTS Variable for Detecting Missing Processes

The MOOSDB, about once per second, posts the variable DB_CLIENTS containing a comma-separated list of clients (other MOOS apps) currently connected to the MOOSDB. When uProcessWatch is configured to watch for all processes, it simply augments the watch list for any process that ever appears on the incoming DB_CLIENTS mail. If the process is then missing at a later reading of this DB_CLIENTS mail, is is considered AWOL.

3.2 Defining the Watch List

The *watch list* is a list of processes, i.e., MOOS apps. Each item on the list will be reported missing if it does not appear in the list of clients shown by the current value of DB_CLIENTS. By default, all clients that ever appear on the DB_CLIENTS list will be added to the watch list. The same is true for all processes named in the Antler configuration block of the mission file. This default can be overridden by the following configuration option:

```
watch_all = false
```

The value of watch_all may also be set to "antler" to indicate that items on the Antler list are to be watched by default, but not those that appear in the DB_CLIENTS list. Likewise it may be set to "dbclients" to indicate that items in the DB_CLIENTS list are to be watched by default, but not those in the Antler list. Processes may be added to the watch list by explicitly naming them in configuration block as follows:

```
watch = process_name[*]
```

A process may be named explicitly, or the prefix of the process may be given, e.g., watch=uXMS*. This will match all processes fitting this pattern, e.g., uXMS_845, uXMS_23.

Processes may be explicitly *excluded* from the watch list with configurations of the following:

```
nowatch = process_name[*]
```

This is perhaps most appropriate when coupled with watch_all=true. If a process is for some reason both explicitly included and excluded, the inclusion takes precedent.

3.3 Reports Generated

There are three reports generated in the variables:

- PROC_WATCH_EVENT
- PROC_WATCH_SUMMARY,
- PROC_WATCH_FULL_SUMMARY

All reports are generated only when there is a change of status in one of the watched processes. The first type of report is generated for each event when a watched process is noted to have connected or disconnected to the MOOSDB. The following are examples:

```
PROC_WATCH_EVENT = "Process [pMarinePID] is noted to be present."
PROC_WATCH_EVENT = "Process [pMarinePID] has died!!!!"}
PROC_WATCH_EVENT = "Process [pMarinePID] is resurected!!!"
```

In the first line above, a process is reported to be present that was never previously on the watch list. In the third line a process that was previously noted to have left the watch list is reported to have returned or been restarted.

The PROC_WATCH_SUMMARY variable will list the set of processes missing from the watch list or report "All Present" if no items are missing. For example:

```
PROC_WATCH_SUMMARY = "All Present"
PROC_WATCH_SUMMARY = "AWOL: pMarinePID,uSimMarine"
```

The PROC_WATCH_FULL_SUMMARY variable will list a more complete and historical status for all processes on the watch list. For example:

```
PROC_WATCH_FULL_SUMMARY = "pHelmIvP(1/0),uSimMarine(1/1),pMarinePID(2/2)"
```

The numbers in parentheses indicate how many times the process has been noted to connect to the MOOSDB over the number of times it has been noted to have disconnected. A report of "(1/0)" is the healthiest of possible reports, meaning it has connected once and has never disconnected.

3.4 Watching and Reporting on a Single MOOS Process

If desired, uProcessWatch may be configured to generate a report dedicated a single MOOS process with the following example configuration:

```
watch = pBasicContactMgr : BCM_OK
```

In this case a MOOS variable, BCM_OK, will be set to either true or false depending on whether the process pBasicContactMgr presently appears on the list of connected clients in listed in DB_CLIENTS.

3.5 A Heartbeat for the Watch Dog

By default uProcessWatch will only post PROC_WATCH_SUMMARY when its value changes. A long stale PROC_WATCH_SUMMARY = "All Present" likely means that everything is fine. It could also mean the uProcWatchSummary process itself died without ever posting anything to suggest otherwise. The user has the configuration option to post PROC_WATCH_SUMMARY every N seconds regardless of whether or not the value has changed:

```
summary_wait = 30  // Summary posted at least every 30 seconds
```

This in effect creates a heartbeat for monitoring uProcessWatch. The default value for this parameter is -1. Any negative value will be interpreted as a request for postings to be made only when the posting value changes. Regardless of the summary_wait setting, the other two reports, PROC_WATCH_EVENT and PROC_WATCH_FULL_SUMMARY, will only be made when the values change.

3.6 Excusing a Process

An application on the watch list may be excused and disconnect from the MOOSDB if it posts to the variable EXITED_NORMALLY with the name of itself. A check is made by uProcessWatch of the *source* of the posting to ensure that message was posted by the exiting process. It's up to the developer of an application to build in the feature declaring a normal exit. An example is the uTimerScript application which has the ability to execute a script of postings to the MOOSDB followed by an exit. In this case, the *status* of application becomes EXCUSED, as shown in Figure 3.

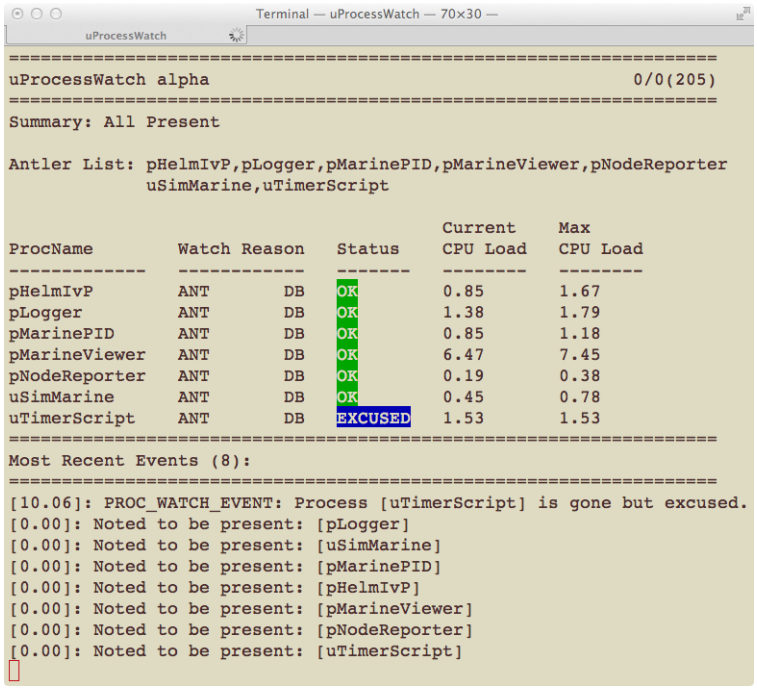

Figure 3: The process uTimerScript has gone missing due to its own intentional exit. Before exiting it posted EXITED_NORMALLY=uTimerScript, and therefore uTimerScript is regarded as excused.

Excusing a missing process is different than choosing to not include it on the watch list. Some applications are, by their nature, designed to disappear at some point. Scoping tools like uXMS, uPokeDB or uHelmScope are examples applications that regularly appear and disappear. They are best excluded entirely from the watch list with the nowatch configuration parameter.

3.7 Allowing Retractions if a Process Reappears

With the addition of appcasting to uProcessWatch, a missing process also triggers an appcast run warning, as shown on the top in Figure 4 below.

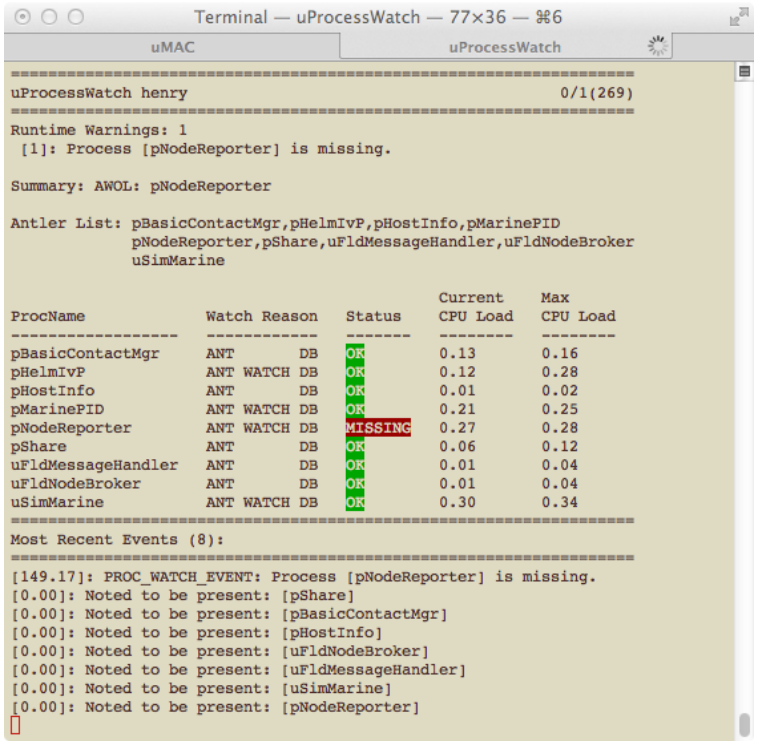

Figure 4: The process pNodeReporter has gone missing. This is noted as a runtime warning at the top, and the MISSING status in the body of the report.

However, there are cases where a process goes missing but then returns, in which case the warning is a distraction. These reasons include.

- The application may actually exit and then restart a short time later.
- The application my be running at a very slow apptick in which case it may be dropped from the DB_CLIENTS list momentarily.
- The application may be missing only because it hasn't launched yet.

To address this, uProcessWatch will allow retractions on appcast run warnings. If the application disappears and reappears, the appcast run warning will disappear. The successive posted values of PROC_WATCH_SUMMARY will show that the process was at some point declared AWOL, but once the process returns, the appcast output will no longer raise attention to the issue.

Of course there are situations where a process that disappears and then reappears is an indication of a problem, not to be swept under the rug. In this case the default behavior of uProcessWatch may be changed by setting allow_retractions to false. Presently this setting cannot be set on a per-process manner.

4 Configuration Parameters of uProcessWatch

The following parameters are defined for uProcessWatch. A more detailed description is provided in other parts of this section. Parameters having default values indicate so in parentheses below.

Listing 4.1: Configuration Parameters for uProcessWatch.

allow_retractions:	If true, run warnings are retracted if a process reappears after disappearing. Legal values: true, false. The default is true.
nowatch:	A process or list of MOOS processes to *not* watch.
post_mapping:	A mapping from one posting variable name to another.
summary_wait:	A maximum amount of time between PROC_WATCH_SUMMARY postings. Negative value indicates posting occurs only when the value changes regardless of elapsed time. Legal values: any numerical value. The default is -1.
watch:	One or more comma-separated MOOS process to watch and report on.
watch_all:	If true, watch all processes that become known either via the DB_CLIENTS list or the Antler list. Legal values: true, false. The default is true.

4.1 An Example MOOS Configuration Block

Listing 2 shows an example MOOS configuration block produced from the following command line invocation:

```
$ uProcessWatch --example or -e
```

Listing 4.2: Example configuration of the uProcessWatch application.

```
ProcessConfig = uProcessWatch
{
  AppTick   = 4
  CommsTick = 4

  watch_all = true   // The default is true.

  watch  = pMarinePID:PID_OK
  watch  = uSimMarine:USM_OK

  nowatch = uXMS*

  allow_retractions = true   // Always allow run-warnings to be
                             // retracted if proc re-appears

  // A negative value means summary only when status changes.
  summary_wait = 10 // Seconds. Default is -1.

  post_mapping = PROC_WATCH_FULL_SUMMARY, UPW_FULL_SUMMARY
}
```

5 Publications and Subscriptions for uProcessWatch

The interface for uProcessWatch, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ uProcessWatch --interface or -i
```

5.1 Variables Published by uProcessWatch

The primary output of uProcessWatch to the MOOSDB is a summary indicating whether or not certain other processes (MOOS apps) are presently connected.

- APPCAST: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility.
- PROC_WATCH_EVENT: A report indicating a particular process has been noted to be gone missing or noted to have (re)joined the list of active processes.
- PROC_WATCH_FULL_SUMMARY: A single string report for each process indicating how many times it has connected and disconnected from the MOOSDB.
- PROC_WATCH_SUMMARY: A report listing all missing processes, or "All Present" if no processes are missing.

The user may also configure uProcessWatch to make a posting dedicated to a particular watched process. For example, with the configuration watch=pNodeReporter:PNR_OK, the status of this process is conveyed in the MOOS variable PNR_OK, set to either true or false depending on whether or not it is present.

The variable name for any posted variable may be changed to a different name with the post_mapping configuration parameter. For example, post_mapping=PROC_WATCH_EVENT, UPW_EVENT will result in events being posted under the UPW_EVENT variable rather than PROC_WATCH_EVENT variable.

5.2 MOOS Variables Subscribed for by uProcessWatch

The following variable(s) will be subscribed for by uProcessWatch:

- APPCAST_REQ: A request to generate and post a new apppcast report, with reporting criteria, and expiration.
- DB_CLIENTS: A comma-separated list of clients currently connected to the MOOSDB, posted by the MOOSDB. Used for detecting missing processes. Section 3.1.
- EXITED_NORMALLY: An indication made by a process, potentially on the watch list, that it has exited normally and should be excused, and not regarded as missing. Section 3.6.
- <PROCNAME>_STATUS: The status string generated for all MOOSApps, containing the current CPU load among other things.

uLoadWatch: Monitoring Application System Load

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/appdocs/app_uloadwatch

Note: Post Release 24.8, uLoadWatch has been modified to publish a "loadcast" request to the apps it is watching. Only these apps will generate _ITER_GAP and _ITER_LEN postings. All other apps will not post this information. This change is in conjunction with the AppCastingMOOSApp superclass, the super class for most apps in the MOOS-IvP release. The motivation of this change is to reduce log file bloat and MOOSDB traffic. Neither reduction was deemed critical, but it was inefficient. These two variable postings, for each app, are essentially only published to be consumed by uLoadWatch. If this app is not even being run in a mission, there is arguably no reason for apps be publishing these variables.

1 Overview

The uLoadWatch application monitors data produced by individual applications and reports a warning when the MOOS community begins to exhibit a strain in the CPU load. Load data is derived solely from two MOOS variables published by participating MOOS apps. For an example application pFooBar, the following two variables are published:

- PFOOBAR_ITER_GAP: indicates the ratio of (a) observed time between invocations to the application's Iterate() method, and (b) the scheduled time between iterations based on the configured AppTick. For example, for an application running at 4 Hz, a gap of 1.0 means things are running normally with 0.25 seconds in between iterations. A gap of 3 indicates that there was 0.75 seconds between iterations - an an idication that the CPU cannot keep up

with the work being done in the iterate loop. This could, however, be due in part to other processes demanding CPU resources.

- PFOOBAR_ITER_LEN: indicates the ratio of (a) observed time between the begin and end of the `Iterate()` loop, and (b) again, the scheduled time between iterations based on the configured AppTick. In this case, a value of 1 is an indication that the work being done in the iterate loop is dangerously close to capacity.

The ITER_GAP variable indicates when strain is actually being exhibited. The ITER_LEN variable indicates the run-up to problems. The uLoadWatch application monitors the situation by registering for the *_ITER_GAP and *_ITER_LEN from all apps and doing three things:

1. Threshold detection. A user configured threshold for ITER_GAP values may be set, with a posting to LOAD_WARNING when it is exceeded.
2. An AppCast run warning will also be posted when the user configured threshold is exceeded. This will bring a critical load situation to the attention of an operator through the existing AppCast mechanisms.
3. An AppCast report is generated reporting the average and maximumn ITER_GAP and ITER_LEN noted thus far. This can be monitored via one of the AppCast Viewers to monitor the load fluctuation.

2 Configuration Parameters for uLoadWatch

The uLoadWatch application may be configured with a configuration block within a MOOS mission file, typically with a `.moos` file suffix. The following parameters are defined for uLoadWatch.

Listing 2.1: Configuration Parameters for uLoadWatch.

breach_trigger:	The number of times a threshold can be breached before a warning is posted. The default is 1 meaning the first offense is forgiven, since it is not uncommon for some apps to have a longer initial gap as they perform one-time start-up work.
near_breach_thresh:	In addition to normal threshold breaches, the uLoadWatch app may also monitor for *near* breaches. A near breach occurs when the normal breach thresh is nearly met. For example, if the threshold for a certain app is 2.0, this means a breach is declared if time between app iterations is greater than 2x the normal gap between iterations. If the near_breach_thresh is set to 1.6, a near breach is declared if the gap reaches $1.6x$ the normal gap.
thresh:	A gap threshold for reporting a LOAD_WARNING. Each threshold line specifies the application name and the gap threshold value.

An Example MOOS Configuration Block

An example MOOS configuration block may be obtained from the command line with the following:

```
$ uLoadWatch --example or -e
```

Listing 2.2: Example configuration of the uLoadWatch application.

```
1  ============================================================
2  pHostInfo Example MOOS Configuration
3  ============================================================
4
5  ProcessConfig = pHostInfo
6  {
7    AppTick   = 4
8    CommsTick = 4
9
10   thresh = app=pHelmIvP, gapthresh=1.5
11   thresh = app=any,      gapthresh=2.0
12
13   near_breach_thresh = 0.9   // default is off
14
15   breach_trigger = 5         // default is 1
16 }
```

3 Publications and Subscriptions for uLoadWatch

The interface for uLoadWatch, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ uLoadWatch --interface or -i
```

3.1 Variables Published by uLoadWatch

The primary output of uLoadWatch is the occasional load warning plus the appcasting report. In Release 20.2, a few additional variables were added to summarize breaches and near breaches.

- APPCAST: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility.
- LOAD_WARNING: A warning indicating an app has been detected reporting an gap greater than the configured threshold. (New, after Release 24.8)
- LOADCAST_REQ: A request, naming another MOOS app, from which ITER_LEN and ITER_GAP information is requested. Such a request will be posted for all apps named in the thresh configurations.
- ULW_BREACH: Posted with value `true`, if and only when a breach has been detected, for any app.
- ULW_BREACH_COUNT: The total amount of breaches detected, for all all apps.
- ULW_BREACH_LIST: The list of all apps for which at least one breach has been detected.
- ULW_NEAR_BREACH: Posted with value `true`, if and only when a *near* breach has been detected, for any app.
- ULW_NEAR_BREACH_COUNT: The total amount of *near* breaches detected, for all all apps.
- ULW_NEAR_BREACH_LIST: The list of all apps for which at least one *near* breach has been detected.

3.2 Variables Subscribed for by uLoadWatch

The uLoadWatch application subscribes all variables ending with the suffix _ITER_GAP and the suffix _ITER_LEN:

- APPCAST_REQ: A request to generate and post a new apppcast report, with reporting criteria, and expiration.
- LOADCAST: A report from an application indicating its most recent observed gap between iterations, and most recent observed iteration duration. (New after Release 24.8)
- *_ITER_GAP: A report from an application indicating its most recent observed gap between iterations.
- *_ITER_LEN: A report from an application indicating its most recent observed itertion duration.

3.3 Command Line Usage of uLoadWatch

The uLoadWatch application is typically launched with pAntler, along with a group of other shoreside modules. However, it may be launched separately from the command line. The command line options may be shown by typing:

```
$ uLoadWatch --help or -h
```

Listing 3.3: Command line usage for the uLoadWatch tool.

```
1  Usage: uLoadWatch file.moos [OPTIONS]
2
3  Options:
4    --alias=<ProcessName>
5        Launch pHostInfo with the given process
6        name rather than pHostInfo.
7    --example, -e
8        Display example MOOS configuration block
9    --help, -h
10       Display this help message.
11   --interface, -i
12       Display MOOS publications and subscriptions.
13   --version,-v
14       Display the release version of pHostInfo.
15
16 Note: If argv[2] is not of one of the above formats
17       this will be interpreted as a run alias. This
18       is to support pAntler launching conventions.
```

4 Usage Scenarios the uLoadWatch Utility

A typical usage of the uLoadWatch utility is shown in Figure 1 below. If the mission is being monitored by a human (vs. an offline simulation), a load warning will present itself in any of the appcast viewing tools, e.g., pMarineViewer, uMACView, or uMAC. The pNodeReporter application also registers for LOAD_WARNING mail and will include the load warning in vehicle node reports. Depending on how pMarineViewer is configured, the vehicle label may indicate a load warning if it occurs.

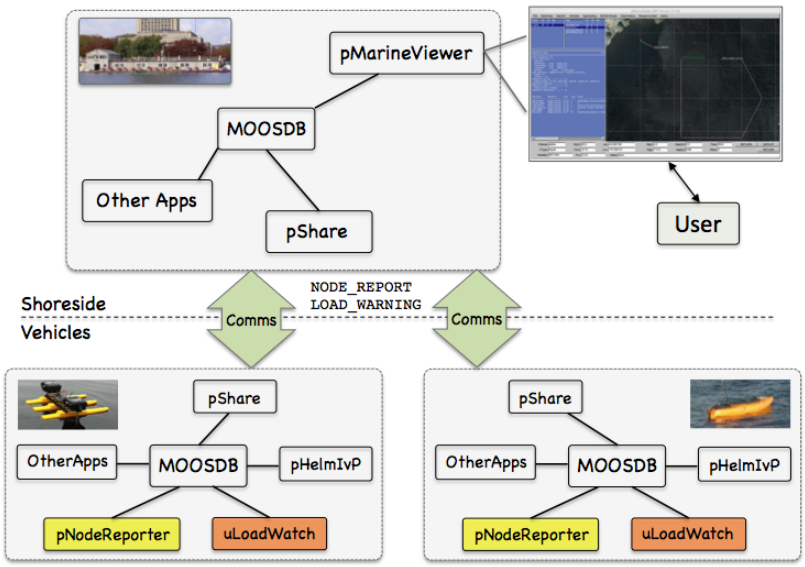

Figure 1: **Typical uLoadWatch Topology:** A shoreside or topside community is receiving information from several deployed vehicles, in the form of node reports, and AppCast reports and warnings. If a load warning is produced, it will (a) show up in any shoreside appcast viewer, (b) show up in node reports sent to the shoreside, and (c) show up in local vehicle alog files.

When running an offline simulation, the output of uLoadWatch is primarily found in the log files.

5 Terminal and AppCast Output

The uLoadWatch application produces some useful information to the terminal on every iteration of the application. An example is shown in Listing 4 below. This application is also appcast enabled, meaning its reports are published to the MOOSDB and viewable from any uMAC application or pMarineViewer. See the document for uMac for more on appcasting and viewing appcasts. The counter on the end of line 2 is incremented on each iteration of uLoadWatch, and serves a bit as a heartbeat indicator. The "0/0" also on line 2 indicates there are no configuration or run warnings detected.

Listing 5.4: Example terminal or appcast output for uLoadWatch.

```
1  ================================================================
2  uLoadWatch henry                                          0/0(129)
3  ================================================================
4  Configured Thresholds:
5    ANY: 2
6    PHELMIVP: 1.5
7
8  Application       AvgGap  MaxGap  AvgLen  MaxLen
9  ----------------  ------  ------  ------  ------
10 PBASICCONTACTMGR  1.01    1.02    0.00    0.02
11 PHELMIVP          1.01    1.04    0.00    0.01
12 PHOSTINFO         1.01    1.01    0.00    0.58
13 PNODEREPORTER     1.01    1.02    0.00    0.01
```

```
14  UFLDNODEBROKER    1.01    1.01    0.00    0.00
15  ULOADWATCH        1.01    1.01    0.00    0.00
16  UPROCESSWATCH     1.00    1.02    0.00    0.00
17  USIMMARINE        1.07    1.15    0.02    0.17
```

The first few lines indicate any user-configured thresholds being applied to incoming reports. Here a LOAD_WARNING will be issued if the pHelmIvP has an iter gap above 1.5, and if any application has a gap over 2.0. The remainder of the report (lines 8-17 here) provide a live update of the average and maximum iter gap and iter length reported for each application reporting.

6 How to Modify a MOOS App to Produce Load Information

All the applications shown in Listing 4 above produce the *_ITER_GAP and *_ITER_LEN information by virtue of being an AppCastingMOOSApp. If you have an application that does not subclass this superclass, it is still pretty easy to add this to your class.

To add the ITER_LEN, and ITER_GAP information, follow the example of the pseudocode below. The code requires some memory between iterations of the timestamp of the prior iteration. So the code requires the additional member variable, added to the YourApp class definition, m_last_iterate_time, initialized to zero in the constructor.

Listing 6.5: Adding appcast output for uLoadWatch.

```
1  YourApp::Iterate()
2  {
3    // Note the time the iterate loop started
4    double start_time = MOOSTime();
5
6
7    (The prior guts of your Iterate loop)
8
9
10   // Note the time the iterate loop ended
11   double end_time = MOOSTime();
12
13   // Only report iter_gap, iter_len if app frequency is > zero
14   double app_freq = GetAppFreq();
15   if(app_freq > 0) {
16      double interval = 1 / app_freq;
17      string app_name = MOOSToUpper(GetAppName());89
18
19      double iter_len = (MOOSTime() - start_time) / interval;
20      Notify(app_name + "_ITER_LEN", iter_len);
21
22      double iter_gap = (start_time - m_last_iterate_time) / interval;
23      if(m_last_iterate_time != 0)
24        Notify(app_name + "_ITER_GAP", iter_gap);
25      m_last_iterate_time = start_time;
26   }
27 }
```